AI Pioneers

CCA-F | Module 4

Lab 4.3

Scaling Output: Batch Processing & Multi-Pass Review

Scenario: A News & Media-Monitoring Pipeline (Helix Robotics coverage)

Module	M4 — Prompt Engineering & Structured Output
Lab type	Hands-on · Self-paced or Instructor-Led · Claude API (Python)
Duration	~45 minutes (hands-on)
Environment	Blue Labs VM · Python 3.9+ · anthropic SDK · `ANTHROPIC_API_KEY`
Sections	S5 (Batch Processing with Claude API) S6 (Multi-Instance & Multi-Pass Architectures)

Lab Objectives

By the end of this lab you will be able to:

- **Use the Message Batches API for large offline workloads** — classify a big set of news mentions asynchronously and collect results by `custom_id`, trading latency for cost and scale.
- **Architect multi-instance processing for throughput** — split a burst of breaking-news headlines across a `ThreadPoolExecutor` so the I/O waits overlap and the run finishes far faster.
- **Apply multi-pass review for higher-quality results** — draft a media briefing, critique it against the team's standards, then refine it; separating generate from judge catches issues a single pass misses.

1. Overview

Some Claude workloads are huge and not time-critical; some are time-critical bursts; some need higher quality than a single shot delivers. This lab walks one media-monitoring pipeline through all three patterns. First you classify a workload of news headlines through the Message Batches API — submit many requests at once, poll until processing ends, then collect results by custom_id. Next you handle a breaking-news burst with a thread pool — many in-flight requests at once, real wall-clock speedup, throttled to your rate limits. Finally you upgrade the morning briefing with a multi-pass review — draft, critique against explicit standards, refine. Each exercise is a self-contained Python script you run against the Claude API. This lab covers Module 4 sections S5 and S6.

SN	Section	What you will do	Core idea
Ex 1	**S5 — Message Batches API**	Submit many `{custom_id, params}` requests, poll until `processing_status == "ended"`, collect by `custom_id`.	Trade latency for cost and scale; results match back by id.
Ex 2	**S6 — Parallel processing**	Run a `ThreadPoolExecutor` over the headlines and compare sequential vs parallel wall-clock time.	I/O-bound waits overlap; throughput rises until you hit rate limits.
Ex 3	**S6 — Multi-pass review**	Draft a briefing, then `critique()` it against standards, then `refine()` to produce a final version.	Separate generate from judge; the critique drives concrete fixes.

2. Scenario

You are building the pipeline for a media-monitoring team that tracks how the press is covering each company on its watchlist. The pipeline has three jobs that pull in different directions. Overnight, thousands of mentions need to be classified for sentiment — accuracy and cost matter, latency does not. During the day, a breaking story can produce a sudden burst — now latency is everything. And every morning the team ships a short briefing summarizing the day's coverage — quality matters most, and one pass is rarely enough.

Your job is to wire each job to the right Claude pattern. You will batch the overnight workload, parallelize the burst, and add a critique-and-refine pass to the briefing — all against the same running example, news coverage of Helix Robotics.

You will:

- Submit a list of `{custom_id, params}` requests through `client.messages.batches.create(...)`, poll `client.messages.batches.retrieve(batch.id)` until `processing_status == "ended"`, and match results back by `custom_id`.
- Implement `run_parallel()` with a `ThreadPoolExecutor` so the headlines are classified concurrently, and measure sequential-vs-parallel wall-clock time.
- Implement `critique()` and `refine()` so the morning briefing flows draft → critique → refined, with the critique step listing concrete problems (not a rewrite).

The three jobs at a glance

```
overnight   -> Message Batches API    (async, cheap, large scale)
breaking    -> ThreadPoolExecutor     (concurrent, fast, rate-limited)
morning     -> Multi-pass review      (draft -> critique -> refine)
```

Why these three together?

Each pattern fits a different cost-vs-latency-vs-quality point. Batches process huge offline workloads at lower cost but make you wait; threads buy you speed when you need it now, capped by rate limits; multi-pass review buys you

quality by letting a second pass see what the first one missed. A pipeline that ships at production scale uses all three: batches for the bulk, threads for the burst, and multi-pass where one shot isn't good enough. Mixing them up — running breaking news through a batch, or burning a thread pool on a job that could wait until morning — wastes either time or money.

3. Pre-requisites

3.1 Skilljar Videos — Complete Before the Session

Course	Lessons to watch	Covers
Building with the Claude API	(no dedicated lesson — built in the lab; reference the Message Batches API docs)	**S5**
Building with the Claude API	Parallelization workflows · Chaining workflows	**S6**

Module 4 uses the Building with the Claude API course on Skilljar. Section S5 has no dedicated lesson — it is built here against the Message Batches API documentation. Complete the S6 lessons above before the session; the instructor will not re-teach the basics — bring questions to the Doubt Session instead.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open https://www.bluelabs.studio/, locate your VM, click Start, then Connect.
- Unzip the lab bundle and enter it:

```
unzip lab-4.3-batch-and-multipass.zip
cd lab-4.3-batch-and-multipass
```

- Create a Python environment and install the Anthropic SDK:

```
python -m venv .venv && source .venv/bin/activate   # Windows:
.venv\Scripts\activate
pip install -r requirements.txt                     # anthropic>=0.40.0
```

- Add your API key and load it into the environment:

```
cp .env.example .env          # paste your key into .env
export $(grep -v '^#' .env | xargs)
```

- Confirm your key and SDK with Exercise 2 — its sequential pass runs immediately on real API calls; the full sequential-vs-parallel comparison works once you implement run_parallel() (small, fast — ~10s each once complete):

```
python exercise_2_parallel.py
```

Model, cost & safety

The scripts read the model from `ANTHROPIC_MODEL` (default `claude-sonnet-4-6`) and the key from `ANTHROPIC_API_KEY`. All three exercises make real API calls. Exercise 1 submits a real batch — usually finishes in minutes but can take longer; if your session ends before it does, re-fetch results later with `--fetch <batch_id>`. Exercise 2's parallel pool default is 5 workers; raise it cautiously — a too-large pool will hit 429 rate-limit errors. Costs are modest (small payloads, short outputs).

4. Exercises

Each exercise is one self-contained file you run. Open the starter, fill in the TODOs, run it. All three exercises make real API calls — Exercise 1 in particular submits a real batch job and waits for it to finish.

Exercise 1: Classify a workload with the Message Batches API (S5) ~15 min

Background

Some workloads don't need an immediate answer — they need a cheap, reliable one. Sentiment-classifying every news mention overnight is the textbook case. The Message Batches API takes a list of requests, processes them asynchronously, and lets you fetch the results when `processing_status` reports `"ended"`. The trade-off is latency: batches usually finish in minutes but can take longer, so you tag every request with a `custom_id` and match results back to your inputs by id.

Task

Open `exercise_1_message_batches.py` and complete two pieces: `build_requests()` (one batch request per headline, each with a unique `custom_id`), and the polling loop inside `main()` (retrieve the batch, print status, break on `"ended"`, sleep between polls, bail out politely past the deadline).

Step 1 — Build the batch requests

Each request is a dict with a `custom_id` and a `params` dict that mirrors the normal `messages.create` call: model, max_tokens, messages. Tag every headline with a stable id so you can join the results back later:

```
def build_requests(headlines):
    return [
        {
            "custom_id": f"headline-{i}",
```

```
            "params": {
                "model": MODEL,
                "max_tokens": 20,
                "messages": [{"role": "user", "content": PROMPT.format(h=h)}],
            },
        }
        for i, h in enumerate(headlines)
    ]
```

Step 2 — Submit and poll until ended

Submit once with `client.messages.batches.create(requests=...)`; then loop: retrieve the batch, print its status and counts, break when `processing_status == "ended"`, sleep, and bail out past the deadline with a hint for re-fetching:

```
deadline = time.time() + 600  # 10 minutes
while True:
    batch = client.messages.batches.retrieve(batch.id)
    print("status:", batch.processing_status, "| counts:", batch.request_counts)
    if batch.processing_status == "ended":
        break
    if time.time() > deadline:
        print("Still processing. Fetch later with --fetch", batch.id)
        return
    time.sleep(10)
```

Step 3 — Collect results by custom_id

Results stream back; each entry's `custom_id` tells you which input it corresponds to. Print succeeded entries; flag the rest by type so failures are visible:

```
for entry in client.messages.batches.results(batch.id):
    if entry.result.type == "succeeded":
        text = "".join(b.text for b in entry.result.message.content
                       if b.type == "text").strip()
        print(f"{entry.custom_id}: {text}")
    else:
        print(f"{entry.custom_id}: ERROR ({entry.result.type})")
```

Step 4 — Run; re-fetch later if needed

```
python exercise_1_message_batches.py

# If your session ends before the batch does, fetch later:
python exercise_1_message_batches.py --fetch <batch_id>
```

Expected: a submitted batch id, status lines as the counts move from processing to succeeded, then one labeled result per input headline (e.g. headline-0: positive, headline-1: negative, ...).

What good looks like

The batch reaches `processing_status == "ended"` within the deadline, every input headline has a result, and the mapping from result back to headline goes through `custom_id` (not result order). If the run hits the deadline, the script exits cleanly and prints the `--fetch <batch_id>` command for later — no work is lost.

Reflection Questions

- When is the Message Batches API the right tool, and when is it the wrong one?
- Why does every request need a `custom_id`? What breaks if you match results back by result order instead?
- The script polls with a 10-minute deadline and prints a re-fetch hint on timeout. Why design the loop this way instead of waiting forever?

Exercise 2: Parallel processing for throughput (S6) ~15 min

Background

When a breaking story hits, you need answers now — not overnight. Classifying eight headlines one-at-a-time waits for each round trip in turn; classifying them through a thread pool overlaps the API waits and finishes in roughly the time of the slowest one. The work is I/O-bound (most of the wall clock is spent waiting on the API), so threads are the right primitive — no GIL contention, and your numbers improve linearly until you hit the API's rate limits.

Task

Open `exercise_2_parallel.py` and implement `run_parallel()`. Use a `ThreadPoolExecutor` to classify the headlines concurrently while preserving input order, then compare sequential vs parallel wall-clock time and print a speedup.

Step 1 — The thread-pool implementation

Use `ThreadPoolExecutor(max_workers=workers)` inside a `with` block and call `.map(...)` so the results come back in input order (no manual id-tagging needed):

```
def run_parallel(client, headlines, workers=5):
    t0 = time.time()
    with ThreadPoolExecutor(max_workers=workers) as pool:
        # .map preserves input order, so results line up with headlines.
        out = list(pool.map(lambda h: classify(client, h), headlines))
    return out, time.time() - t0
```

Step 2 — Run and compare

```
python exercise_2_parallel.py
```

Expected: with 8 headlines and `workers=5`, parallel typically finishes ~3–5× faster than sequential on this size. The exact ratio depends on per-request latency and your rate limits.

Step 3 — Tune workers to your rate limits

Raising `workers` beyond your account's per-minute request limit will produce 429 errors. A safe starting point for this lab is 5; for production work, size the pool to your rate budget and back off gracefully on 429.

> **What good looks like**
>
> Parallel results match the sequential results item-for-item (`.map` preserves order), and the wall-clock drops from a multi-second serial loop to roughly the time of one request. The printed speedup is typically 3–5× at `workers=5`; if it isn't, suspect a too-small pool, an unusually slow request, or a 429 you didn't see.

Reflection Questions

- Why are threads (not processes or async) the right primitive for this workload?
- What stops the speedup from growing linearly with `workers`? Where does the wall come from?
- If you were sizing this pool for production, what number would you set `workers` to, and how would you decide?

Exercise 3: Multi-pass review for higher quality (S6) ~15 min

Background

A morning briefing has to be balanced, specific, and short. A single-shot draft often misses one of those — too long, leans positive, buries the lede. A second pass that only critiques (does not rewrite) against explicit standards produces concrete fixes; a third pass applies them. Separating generate from judge catches what self-review doesn't, because the critique pass has fresh attention on the output rather than defending what it just wrote.

Task

Open `exercise_3_multipass.py` and implement `critique()` and `refine()`. The critique step must list concrete problems against the `STANDARDS` — not rewrite. The refine step must address every point and return only the final briefing text.

Step 1 — The standards the briefing must meet

```
STANDARDS = (
    "Briefing standards: lead with the single most important development; balance "
    "positive and negative coverage fairly; be specific (numbers, who/what); stay "
    "neutral in tone; no speculation beyond the headlines; keep it under 120
words."
)
```

Step 2 — Critique (list problems, don't rewrite)

Send the standards, the headlines, and the draft, and ask for short, actionable bullets. The **do NOT rewrite** instruction matters — it keeps the step focused on judging, so the next step has a concrete checklist to fix:

```
def critique(client, headlines, draft_text):
    joined = "\n".join(f"- {h}" for h in headlines)
    prompt = (
        f"{STANDARDS}\n\n"
        f"Headlines:\n{joined}\n\n"
        f"Draft briefing:\n{draft_text}\n\n"
        "Critique the draft against the standards above. List specific, actionable
"
        "problems as short bullets (e.g. buries the regulatory probe, omits the "
        "12% figure, too long, leans positive). Do NOT rewrite — only list issues."
    )
    return ask(client, prompt)
```

Step 3 — Refine (apply the critique, return only the briefing)

Send the standards, headlines, draft, and critique, and ask for a rewrite that addresses every point. Tell the model to **output only the final briefing** so you don't have to parse around an explanation:

```
def refine(client, headlines, draft_text, critique_text):
    joined = "\n".join(f"- {h}" for h in headlines)
    prompt = (
        f"{STANDARDS}\n\n"
        f"Headlines:\n{joined}\n\n"
        f"Draft briefing:\n{draft_text}\n\n"
        f"Critique to address:\n{critique_text}\n\n"
        "Rewrite the briefing so it fixes every point in the critique and meets the
"
        "standards. Output only the final briefing text."
    )
    return ask(client, prompt)
```

Step 4 — Run and compare draft vs refined

```
python exercise_3_multipass.py
```

Expected output has three labelled blocks: DRAFT (the first pass), CRITIQUE (a short bullet list of issues — too long, omits the 12% figure, buries the probe, etc.), and REFINED (a tighter, balanced, ≤120-word briefing that addresses every bullet).

What good looks like

The CRITIQUE is a short, concrete bullet list — not prose, not a rewrite. The REFINED briefing visibly addresses every bullet: it leads with the single biggest item, balances positive and negative coverage, names specific figures, stays under 120 words, and doesn't speculate beyond the headlines. If the refined version looks almost identical to the draft, your critique was too vague; if it drops accurate facts, the refine prompt isn't constraining the rewrite enough.

Reflection Questions

- Why use two separate calls (critique, then refine) instead of one prompt that asks the model to both review and rewrite?
- The critique prompt explicitly says "Do NOT rewrite — only list issues." What goes wrong if you drop that instruction?
- This lab uses the same model for all three passes. When would you want a **different** model (or a different instance) to run the critique?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking back.

#	Question
1	When is the Message Batches API the right choice over real-time calls — and when is it the wrong one?
2	Why must every batch request have a `custom_id`, and what is the polling contract (which status, how often, when to give up)?
3	Why is a thread pool (not a process pool, not `asyncio`) the right primitive for parallel Claude calls, and what caps the speedup?
4	Why split a multi-pass review into separate generate/critique/refine calls instead of asking one prompt to do all three?
5	Which of the three patterns (batch / parallel / multi-pass) fits which job in the pipeline, and what does mixing them up cost you?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Using batches for real-time work.	Batches trade latency for cost and scale; a user-facing interaction will time out long before the batch ends. Use real-time calls (or a thread pool) for anything synchronous; reserve batches for overnight or backfill jobs.
Matching results back by order, not by `custom_id`.	Batch results arrive in completion order, not submission order. Tag every request with a stable `custom_id` and join results back through it.
Polling forever with no deadline.	A stuck or slow batch will hang your script and burn CI minutes. Cap the loop with a deadline and print a `--fetch <batch_id>` hint so the run can resume later without re-submitting.
Cranking `workers` past your rate limit.	Beyond the API's per-minute requests-per-minute (or tokens-per-minute) limit, additional workers just generate 429s. Start at 5, raise cautiously, and handle 429 with backoff.

Mistake	Why it matters and what to do instead
Reaching for a process pool instead of threads.	This work is I/O-bound; the GIL doesn't matter. Threads share the SDK client and have far lower overhead. Use `ThreadPoolExecutor`, not `ProcessPoolExecutor`.
Asking critique to rewrite.	If the critique step rewrites the draft, you collapse generate and judge into one and lose the value of the separation. Tell it "list issues, do NOT rewrite" — keep the rewrite for the refine step.
Self-review for high-stakes output.	The model that wrote the draft is biased to defend it. For high-stakes critique, use a fresh instance (or a different model) so the review has unbiased attention on the text.

6. Quick Reference

6.1 Message Batches: submit, poll, collect

```
# Each request: {custom_id, params}.  params mirrors messages.create().
requests = [
    {"custom_id": f"headline-{i}",
     "params": {"model": MODEL, "max_tokens": 20,
                "messages": [{"role":"user","content": PROMPT.format(h=h)}]}}
    for i, h in enumerate(headlines)
]

batch = client.messages.batches.create(requests=requests)
while True:
    batch = client.messages.batches.retrieve(batch.id)
    if batch.processing_status == "ended": break
    time.sleep(10)

for entry in client.messages.batches.results(batch.id):
    if entry.result.type == "succeeded":
        ...   # use entry.custom_id to join back to your input
```

6.2 ThreadPoolExecutor for I/O-bound bursts

```
from concurrent.futures import ThreadPoolExecutor

def run_parallel(client, headlines, workers=5):
    with ThreadPoolExecutor(max_workers=workers) as pool:
        # .map preserves input order — results line up with headlines.
        return list(pool.map(lambda h: classify(client, h), headlines))

# Tune `workers` to your rate limit. Too many => 429s.
```

6.3 Multi-pass: draft → critique → refine

```
draft   = ask(client, f"Write a briefing from these headlines:\n{joined}")
critique = ask(client,
    f"{STANDARDS}\n\nHeadlines:\n{joined}\n\nDraft:\n{draft}\n\n"
    "List specific, actionable problems. Do NOT rewrite — only list issues.")
refined = ask(client,
    f"{STANDARDS}\n\nHeadlines:\n{joined}\n\nDraft:\n{draft}\n\n"
    f"Critique to address:\n{critique}\n\n"
    "Rewrite to fix every point. Output only the final briefing text.")
```

6.4 Which pattern for which job?

```
# Cost vs Latency vs Quality — pick the right pattern:
#
#   overnight bulk    -> Message Batches API    (cheap, async, scales)
#   breaking-news     -> ThreadPoolExecutor     (fast, rate-limited)
#   morning briefing -> Multi-pass review      (higher quality)
#
# Wrong fit costs you either time, money, or quality.
```

6.5 File Inventory

File	Section	Purpose
`exercise_1_message_batches.py`	**S5**	Build batch requests with `custom_id`; submit, poll, collect by id. Supports `--fetch <id>` to retrieve later.
`exercise_2_parallel.py`	**S6**	Implement `run_parallel()` with a `ThreadPoolExecutor`; compare sequential vs parallel wall-clock.
`exercise_3_multipass.py`	**S6**	Implement `critique()` and `refine()`; print draft, critique, refined briefing.
`.env.example, requirements.txt`	**setup**	API key template and the `anthropic` SDK dependency.

6.6 Further Reading

- Message Batches API: https://docs.claude.com/en/docs/build-with-claude/batch-processing
- Rate limits (sizing your parallel pool): https://docs.claude.com/en/api/rate-limits
- Skilljar — Building with the Claude API: Parallelization workflows; Chaining workflows (S6)
- Section S5 has no dedicated Skilljar lesson — it is built in the lab against the Message Batches docs above.